

K3H4 Refactoring Report

Executive summary

The conversation can be refactored into a coherent analytical model by treating its topics as layers of a single pipeline rather than as disconnected metaphors. At the core sits **K-means** as a centroidal partitioning method; its geometric interpretation is a **Voronoi partition**; its local scale can be summarized by **inscribed hyperspheres**; its iteration can be written as a **fixed-point operator in lambda-calculus style**; its numerical realization depends on **hardware, byte order, and floating-point interchange**; and its “harmonic” extension is best made rigorous through **spectral clustering and graph Laplacians** rather than through free-form analogy alone. MacQueen’s original 1967 paper explicitly described “k-means” as a process for partitioning an N -dimensional population into k sets with low within-class variance, and also described the now-classic mean/reassign iteration that lowers variance and converges in finitely many steps. Voronoi diagrams provide the exact geometric picture of nearest-centroid assignment, while weighted Voronoi and power-diagram constructions show how radius-aware assignment changes that geometry. ¹

For this report, I define **K3H4** as a four-block model whose name encodes both geometry and systems concerns: **K3** denotes the three geometric state variables retained per cluster—**centroid, cell, and core radius**—and **H4** denotes the four model blocks—**K-means partitioning, hypersphere geometry, harmonic or spectral analysis, and hardware or endianness control**. The decisive analytical point is that the weighted score

$$s_i(x) = \frac{\|x - c_i\|}{r_i}$$

turns the assignment step into a **multiplicatively weighted Voronoi diagram** (MWVD) with weights r_i , while the alternative score

$$p_i(x) = \|x - c_i\|^2 - r_i^2$$

turns it into a **power diagram**. Those are related but not identical geometries: MWVD boundaries are Apollonian spheres and may yield regions with holes, whereas power-diagram boundaries are hyperplanes and the cells are polyhedral. That distinction matters for both theory and implementation. ²

The report’s main recommendation is therefore pragmatic and rigorous. If the goal is a compact scale-aware refactoring of Lloyd’s method, use a **K3H4 fixed-point loop** with radius-aware assignment, mean updates, explicit radius closure, and strict serialization rules. If the goal is interpretable convex cells and stable point location, prefer the **power-diagram variant**. If the data are graph-like, non-convex, manifold-shaped, or affinity-defined, move the geometry into a **spectral embedding first**, then run the K3H4 clustering stage there. Finally, treat endianness as a first-class model layer: the geometry is only as correct as the byte-to-float decoding that creates the coordinates in the first place. Python’s `struct` module, NumPy’s byte-order controls, XDR, Arrow, and Parquet all show that the wire format must specify byte order explicitly. ³

Conversation recap and conceptual map

The conversation moved through six ideas that now fit naturally into one model. **Lambda calculus** supplied the language of recursion and fixed points. **K-means** supplied the base optimization loop. **Voronoi cells** supplied the geometric partition induced by current centroids. **Inscribed hyperspheres** supplied a cluster-local scale parameter, namely a radius attached to each centroid. **Endianness** supplied the systems layer that determines whether the decoded coordinates are even numerically meaningful. **Harmonic or spectral analogies** suggested that cluster geometry can also be viewed through eigenmodes of a graph Laplacian, which is the rigorous version of the “harmonic” idea. MacQueen’s original framing, the Voronoi survey literature, weighted Voronoi and power-diagram literature, and spectral-clustering papers together make those links mathematically clean. 4

A succinct concept map is:

Conversation concept	Role in the K3H4 refactor
Lambda calculus	Encodes the iterative clustering loop as a fixed-point operator
K-means	Supplies the centroidal assignment/update baseline
Voronoi cells	Geometric picture of nearest-center assignment
Inscribed hyperspheres	Per-cluster scale r_i derived from cell geometry
Endianness	Byte-to-float representation layer that precedes all geometry
Harmonic or spectral view	Embeds data via Laplacian eigenmodes when Euclidean partitions are insufficient

That mapping is not merely mnemonic. In standard Lloyd-style K-means, each point is assigned to its nearest centroid, which is equivalent to partitioning space into Voronoi cells generated by those centroids. In the weighted refactor, the score $\|x - c_i\|/r_i$ changes the cells from ordinary Voronoi regions to multiplicatively weighted ones, while $\|x - c_i\|^2 - r_i^2$ yields a power diagram instead. Spectral clustering then appears as an upstream harmonic layer: rather than partitioning the original coordinates, one partitions the rows of an eigenvector embedding of a graph Laplacian. 5

A schematic picture of “Voronoi cells with centered inscribed spheres” is:

Voronoi cells with centered inscribed spheres

The shared boundary is a bisector.
For an unweighted Voronoi cell centered at c_i ,

the largest sphere centered at c_i and contained in its cell has radius $r_i = 1/2 \min_{j \neq i} ||c_i - c_j||$.

This formula follows directly from the ordinary Voronoi construction: each cell boundary is a bisector between two sites, so the nearest boundary to c_i lies halfway to its nearest neighboring centroid. That radius is therefore the largest centered insphere guaranteed to lie inside the current unweighted cell. Once that same radius is reused as a multiplicative weight, however, the geometry changes: the radius was computed from the old unweighted cells, but it influences the next **weighted** cells. This geometric refactoring is the key conceptual bridge in the entire conversation. ⁶

K3H4 model

I define **K3H4** as follows:

$$\mathbf{K3H4} = \underbrace{\text{K-means partitioning}}_{\text{block 1}} + \underbrace{\text{K-cell hypersphere geometry}}_{\text{block 2}} + \underbrace{\text{Harmonic or spectral embedding}}_{\text{block 3}} + \underbrace{\text{Hardware or ...}}_{\text{block 4}}$$

The “K3” mnemonic refers to the three geometric state variables retained per cluster:

$$(c_i, V_i, r_i), \quad i = 1, \dots, k,$$

where c_i is the centroid, V_i is its current cell, and r_i is a core radius. The “H4” mnemonic refers to the four model blocks above. This is an assumed but internally consistent reading of the user’s requested acronym. ⁷

The model state is

$$\Sigma = (B, H, X, W, Z, C, R, A),$$

with the following data structures:

$$\begin{aligned} B &\in \{0, 1\}^* \text{ bytes}, & H &= (\text{dtype}, \text{byteorder}, \text{alignment}, \text{device}), \\ X &\in \mathbb{R}^{n \times d} \text{ decoded features}, & W &\in \mathbb{R}^{n \times n} \text{ or sparse affinity graph}, \\ Z &\in \mathbb{R}^{n \times m} \text{ embedding, with } Z = X \text{ if no spectral layer is used,} \\ C &= (c_1, \dots, c_k) \in \mathbb{R}^{k \times m}, & R &= (r_1, \dots, r_k) \in \mathbb{R}_{>0}^k, \\ A &: \{1, \dots, n\} \rightarrow \{1, \dots, k\}. \end{aligned}$$

The decode layer is

$$X = \text{Decode}_H(B),$$

where H determines byte order and dtype. In the spectral branch, one constructs a similarity graph W , degree matrix D , and Laplacian such as

$$L = D - W, \quad L_{\text{sym}} = I - D^{-1/2} W D^{-1/2},$$

then takes the first k eigenvectors and clusters the row vectors of the resulting embedding. That is exactly the standard spectral-clustering recipe in the tutorial literature and in practice-oriented documentation. 8

The baseline K-means objective in the embedded space is

$$J_{\text{kmeans}}(C, A \mid Z) = \sum_{i=1}^k \sum_{z_t: A(t)=i} \|z_t - c_i\|_2^2.$$

MacQueen described the method as producing partitions with low within-class variance, and the Lloyd-style mean/reassign iteration lowers that variance and converges in finitely many steps. 9

The **radius-aware K3H4 objective** for fixed radii is

$$J_{\text{K3H4}}(C, A \mid Z, R) = \sum_{i=1}^k \frac{1}{r_i^2} \sum_{z_t: A(t)=i} \|z_t - c_i\|_2^2, \quad r_i > \varepsilon.$$

For fixed R , the assignment

$$A(t) = \arg \min_i \frac{\|z_t - c_i\|_2}{r_i}$$

is equivalent to minimizing $\|z_t - c_i\|_2^2 / r_i^2$, and the centroid update remains the arithmetic mean because $1/r_i^2$ is constant within cluster i . Thus the pair (U_A, U_C) remains a Lloyd-like alternating minimization for fixed radii. The extra radius step is what turns the method from pure coordinate descent into a **fixed-point geometric closure**. 10

The required update equations are therefore:

$$c_i^{(t+1)} = \frac{1}{|S_i^{(t)}|} \sum_{z_t \in S_i^{(t)}} z_t, \quad S_i^{(t)} = \{z_t : A^{(t)}(t) = i\},$$

$$r_i^{(t+1)} = \frac{1}{2} \min_{j \neq i} \|c_i^{(t+1)} - c_j^{(t+1)}\|_2,$$

optionally stabilized by damping,

$$r_i^{(t+1)} = \max\left(\varepsilon, \alpha r_i^{(t)} + (1 - \alpha) \frac{1}{2} \min_{j \neq i} \|c_i^{(t+1)} - c_j^{(t+1)}\|_2\right), \quad 0 \leq \alpha < 1.$$

The weighted assignment rule requested by the user is

$$s_i(x) = \frac{\|x - c_i\|_2}{r_i}, \quad A(x) = \arg \min_i s_i(x).$$

This defines cells

$$V_i^\times = \left\{ x : \frac{\|x - c_i\|_2}{r_i} \leq \frac{\|x - c_j\|_2}{r_j} \forall j \right\},$$

which are exactly **multiplicatively weighted Voronoi cells** with weights r_i . Their boundaries satisfy

$$\frac{\|x - c_i\|_2}{r_i} = \frac{\|x - c_j\|_2}{r_j},$$

which are Apollonian spheres when $r_i \neq r_j$ and hyperplanes when $r_i = r_j$. In the computational-geometry literature, MWVD regions may contain holes and can be much more complex than ordinary Voronoi cells.

11

The closest additive analogue is the **power-diagram** score

$$p_i(x) = \|x - c_i\|_2^2 - r_i^2.$$

Power diagrams minimize this quantity over sites, and their bisectors are hyperplanes because

$$\|x - c_i\|_2^2 - r_i^2 = \|x - c_j\|_2^2 - r_j^2$$

reduces to a linear equation in x . Aurenhammer's power-diagram paper defines this geometry as the Voronoi diagram under the "power" quantity $\|x - z_i\|^2 - r_i^2$. This is the best choice when convex cells, clean point-location, and stable diagram construction matter more than multiplicative scale normalization.

12

In lambda-calculus style, using standard syntactic sugar for tuples and conditionals, the fixed-point form of the method is:

```
Y ≡ λf.(λx.f (x x)) (λx.f (x x))

Assign ≡ λC.λR.λZ.
  map (λz. argmin_i (norm(z, c_i) / max(r_i, ε))) Z

UpdateC ≡ λA.λZ.
  map (λi. mean (filter_i A Z)) I_k

UpdateR ≡ λC.
  map (λi. 0.5 * min_{j≠i} norm(c_i, c_j)) I_k

Embed ≡ λX. if spectral then EigEmbed(X) else X

Step ≡ λΣ.
  let X = Decode_H (Bytes Σ) in
  let Z = Embed X in
  let A = Assign (C Σ) (R Σ) Z in
  let C' = UpdateC A Z in
  let R' = UpdateR C' in
  ⟨Bytes Σ, H Σ, X, W(X), Z, C', R', A⟩
```

```

K3H4  $\equiv$   $\lambda$ self.  $\lambda$  $\Sigma$ .
  if Converged  $\Sigma$  then  $\Sigma$  else self (Step  $\Sigma$ )

```

This is only an encoding sketch, not a full Church encoding of vectors and matrices, but it captures the right fixed-point semantics. Cornell lecture notes give the standard untyped Y -combinator exactly in this form.

13

Algorithms, complexity, and variant comparison

A practical pseudocode version is:

```

def k3h4(byte_buffer, hardware_contract, k, spectral=False,
        max_iter=100, tol=1e-5, eps=1e-8, damping=0.5):
    X = decode(byte_buffer, hardware_contract)      # endian-aware
    Z = spectral_embed(X, k) if spectral else X

    C = init_centroids(Z, k)                        # e.g. k-means++
    R = init_radii_from_centroids(C, eps)           # 0.5 * nearest-centroid
    distance

    for _ in range(max_iter):
        A = argmin_over_i(lambda z, i: norm(z - C[i]) / max(R[i], eps), Z)

        C_new = [mean(Z[A == i]) if any(A == i) else reinit_center(Z, C, A, i)
                  for i in range(k)]

        R_geo = [0.5 * min(norm(C_new[i] - C_new[j]) for j in range(k) if j !=
i)
                  for i in range(k)]

        R_new = [max(eps, damping * R[i] + (1 - damping) * R_geo[i])
                  for i in range(k)]

        if max(norm(C_new[i] - C[i]) for i in range(k)) < tol \
            and max(abs(R_new[i] - R[i]) for i in range(k)) < tol:
            C, R = C_new, R_new
            break

        C, R = C_new, R_new

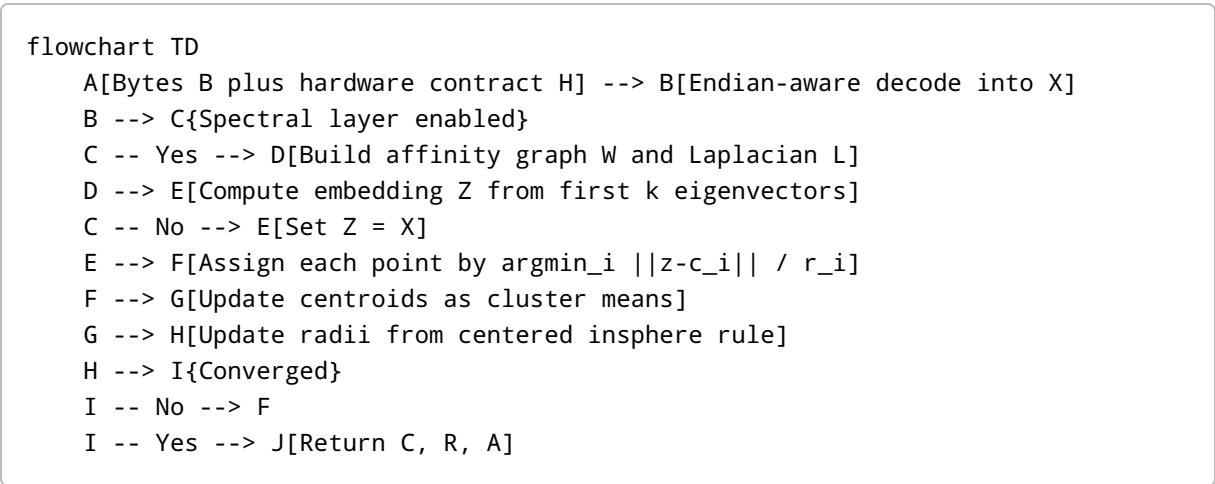
    return C, R, A

```

This pseudocode extends the ordinary assignment/mean loop by a radius-closure update. If the spectral option is on, the loop acts on Z , the eigenvector embedding, rather than the raw coordinates. That is

exactly how mainstream spectral clustering is formulated: compute a low-dimensional embedding from the affinity graph, then cluster the row vectors in that embedding, often by K-means. ¹⁴

The update loop can be visualized as:



From a complexity perspective, the ordinary assignment/update loop is still the dominant cost for centroidal clustering. In scikit-learn's K-means documentation, the algorithm is described as the nearest-centroid assignment followed by a mean update, repeated until the centroids stabilize; the implementation parallelizes these steps over chunks of samples. The added K3H4 radius closure costs $O(k^2)$ per iteration if done by all-pairs centroid distances, so the total per-iteration complexity is naturally

$$O(nkd + k^2)$$

in d dimensions. If the spectral layer is used, the dominant cost shifts to graph construction and eigensolution; sparse affinity matrices make spectral clustering much more practical. Exact MWVD materialization is usually unnecessary and can be expensive: in the plane, MWVDs can have $\Omega(n^2)$ size, and their regions may contain holes. ¹⁵

The four main variants compare as follows:

Variant	Representation	Assignment rule	Update rule	Invariants or geometry	Complexity	Robustness
Standard K-means ¹⁶	Centroids c_i	$\arg \min_i \ x - c_i\ _2^2$	Mean of assigned points	Ordinary Voronoi partition; within-cluster variance decreases under the Lloyd cycle	Iterative nearest-center cost dominates	Good on compact roughly isotropic clusters; weaker on unequal-variance or non-convex data

Variant	Representation	Assignment rule	Update rule	Invariants or geometry	Complexity	Robustness
K-means plus inscribed hyperspheres 17	Centroids plus radii r_i	$\arg \min_i \ x - c_i\ _2 / r_i$ or power score $\arg \min_i \ x - c_i\ _2^2 - r_i^2$	Mean update plus radius closure	MWVD if multiplicative; power diagram if additive-square; radius is centered insphere of prior unweighted cell	$O(nkd + k^2)$ per iteration for score-based use; explicit MWVD construction can become much worse	Better scale normalization; sensitive to tiny radii and to whether multiplicative or power geometry is chosen
Spectral clustering 18	Similarity graph, Laplacian, eigenvector embedding	K-means after embedding	Eigen solve plus centroidal clustering in eigenspace	Graph-cut relaxation; normalized versions have stronger consistency guarantees than unnormalized ones	Dominated by graph construction and eigensolve; sparse graphs help substantially	Strong for non-convex and manifold-shaped clusters; sensitive to graph construction and kernel parameters
Gaussian mixture model 19	Means, covariances, mixture weights	Soft responsibilities	EM updates of means, covariances, weights	Probabilistic partition, not Voronoi cells; generalizes spherical hard assignments	Higher than K-means because density evaluation and covariance updates are required	Stronger on elliptical and heteroscedastic data; more parameters and more numerical sensitivity

A rigorous caution belongs here. The K3H4 loop is not, in general, optimizing a single classical objective all the way through, because the radius closure

$$r_i \leftarrow \frac{1}{2} \min_{j \neq i} \|c_i - c_j\|$$

is a geometric rule, not the exact minimizer of a standard smooth penalty. The assignment step and centroid step are coherent for fixed radii, but the radius step is a **refactoring hypothesis**: it injects local cell scale into the next iteration. That is why the model should be described as a **fixed-point iteration** rather than as a clean convex optimization scheme. The distinction is conceptually important and faithful to the original conversation. 20

Systems and numerical implementation notes

The hardware layer of K3H4 is not incidental. It is the stage that decides whether the bytes on disk or on the wire become the intended floating-point coordinates before any distances, radii, Voronoi cells, or eigenvectors are computed. Python's `struct` documentation makes the essential rule explicit: `<` means little-endian, `>` means big-endian, `!` means network byte order, which is big-endian, and one should not assume that the native layout of the current machine matches the external format. That is exactly why endianness belongs inside K3H4 as a named model block. ²¹

The safest implementation pattern is therefore to **decode explicitly, normalize once, and only then cluster**. NumPy's byte-swapping guide distinguishes two operations: changing the dtype's byte-order interpretation via `newbyteorder()`, and changing the underlying bytes via `byteswap()`. It also warns that scalars do not preserve byte-order information and that some operations convert arrays to native byte order. In other words, if the external buffer and the dtype disagree, you can get numerically wrong values that still look superficially valid. In a clustering pipeline, that is catastrophic because every distance and centroid update downstream is then wrong. ²²

A safe Python sketch is:

```
import struct
import numpy as np

# Suppose the wire format is big-endian float32
vals = struct.unpack(f'>{n*d}f', buf)
X = np.asarray(vals, dtype=np.float32).reshape(n, d)

# Or, with a directly viewed buffer:
X_be = np.frombuffer(buf, dtype='>f4').reshape(n, d)
X = X_be.astype(np.float32, copy=False) # normalize to native float32
```

This is consistent with Python's explicit byte-order controls and with NumPy's documented byte-order conversions. ²³

For interchange formats, the serialization layer must be explicit as well. XDR is a canonical example of a big-endian external representation and specifies IEEE floating-point encodings with ordered bytes in the stream. By contrast, Apache Arrow is little-endian by default and even records schema endianness so that readers can reject mismatches, while Parquet stores native scalars in little-endian form and encodes floating-point values in IEEE format. These are practical examples of the same general lesson: **the clustering math is independent of endianness, but the data pipeline is not**. ²⁴

On CPU and GPU, layout and alignment affect performance visibly. Intel's vectorization guidance emphasizes aligned allocation and the need to tell the compiler when memory accesses are aligned; the article uses 64-byte alignment as the practical target for AVX-512-oriented performance-critical arrays. NVIDIA's CUDA best-practices guide similarly stresses coalesced global-memory access: on recent devices, contiguous accesses by a warp are serviced via as few 32-byte transactions as possible, and misalignment

creates extra transactions. For K3H4, that implies a simple implementation rule: keep feature matrices, centroids, and radii in contiguous arrays; separate hot numeric kernels from serialization; and prefer memory layouts that let vector lanes or warps read adjacent floating-point values. ²⁵

The practical endianness tests I recommend are straightforward. First, maintain **golden wire-format fixtures** for both little-endian and big-endian payloads. Second, include sentinel values such as 1.0, negative values, infinities, and NaNs in those fixtures so that decode failures surface immediately. Third, test both “reinterpret only” and “byteswap plus reinterpret” paths, because NumPy documents those as distinct operations. Fourth, run a **cross-platform reproducibility test**: decode, cluster, serialize the resulting centroids and radii, and verify that a second machine with explicitly opposite wire-order settings reproduces the same decoded numeric arrays before clustering. Those tests are engineering recommendations, but they are directly motivated by the official tooling and format rules above. ²⁶

Harmonic and spectral interpretation

There are two different senses in which “harmonic” can enter this model, and it is useful to keep them separate. The first is a **heuristic cavity analogy**: once each cluster has a radius r_i , one can attach a characteristic frequency scale

$$\omega_i = \frac{\kappa}{r_i + \varepsilon}, \quad f_i = \frac{\omega_i}{2\pi},$$

for some application-dependent constant κ . In that analogy, smaller inscribed spheres correspond to higher characteristic frequencies and larger ones to lower frequencies. This is not a theorem about K-means; it is a design lens that turns local geometric scale into a spectral or resonance parameter. Because it scales like $1/r_i$, it is internally consistent and often helpful for interpretation.

A tiny normalized example is:

Radius r	Frequency scale κ/r
0.5	2κ
1	κ
2	$\kappa/2$

The second—and rigorous—sense of “harmonic” is the **spectral-clustering** one. There the key object is not a sphere radius but the graph Laplacian. In Luxburg’s tutorial, spectral clustering is formulated by building a similarity graph, computing the first k eigenvectors of a Laplacian, and then clustering the row vectors of that eigenvector matrix. The same tutorial states that the multiplicity of the zero eigenvalue equals the number of connected components, and it recommends normalized formulations over the unnormalized one because the normalized methods enjoy stronger consistency behavior. Shi and Malik’s normalized-cuts paper likewise connects the problem to a generalized eigenvalue system on the graph. This is the proper mathematical home of the harmonic intuition. ²⁷

From that perspective, the heuristic “Voronoi cell as resonant cavity” picture and the rigorous “graph Laplacian as harmonic operator” picture can be combined cleanly. The centered insphere radius r_i

measures **local geometric scale** inside a cluster. The low-frequency Laplacian eigenvectors measure **global smooth variation** over the similarity graph. A useful K3H4 interpretation is therefore:

$$\text{local geometry} \longleftrightarrow r_i, \quad \text{global harmonic structure} \longleftrightarrow \text{eigenvectors of } L.$$

That is why a spectral-first K3H4 pipeline is often preferable on manifold or graph-structured data: the embedding resolves the global harmonic organization first, and the centroid-plus-radius layer then captures local scale inside that embedding. ¹⁸

A good rule of thumb is this. Prefer ordinary Euclidean K3H4 when clusters are compact, roughly convex, and meaningfully described by local scale normalization. Prefer **spectral K3H4** when clusters are non-convex, linked by affinity graphs, or better separated by diffusion structure than by raw Euclidean distance. Luxburg explicitly notes that spectral clustering succeeds on structures such as intertwined spirals where K-means fails because K-means produces convex sets in the ambient space. ²⁸

Recommended experiments and open questions

A concise experimental program follows directly from the model. First, compare **standard K-means**, **multiplicative K3H4**, **power-diagram K3H4**, **spectral clustering**, and **GMM** on four canonical data families: isotropic blobs, unequal-size or unequal-variance blobs, anisotropic Gaussians, and non-convex manifolds such as rings or spirals. Second, run each method both in raw space and, where appropriate, in a spectral embedding. Third, run the full pipeline under controlled little-endian and big-endian serialization scenarios to verify that results are invariant after explicit decoding but diverge under intentionally wrong decoding. Those experiments test all four K3H4 blocks at once. ²⁹

For evaluation, use both **intrinsic** and **extrinsic** criteria. Intrinsic metrics should include inertia or weighted distortion, the Silhouette Coefficient, Davies–Bouldin, and Calinski–Harabasz. Scikit-learn’s metric documentation states that higher silhouette values indicate better-defined clusters, lower Davies–Bouldin values indicate better separation relative to dispersion, and Calinski–Harabasz is a variance-ratio criterion. If labels are available, add adjusted Rand index or adjusted mutual information. These should be complemented by **stability metrics**: seed sensitivity, platform sensitivity, and decode reproducibility across byte orders. ³⁰

The main recommendation is to adopt **two K3H4 modes**, not one. Use a **multiplicative mode**

$$s_i(x) = \|x - c_i\|/r_i$$

when you want a normalized notion of “distance in units of local cluster radius.” Use a **power mode**

$$p_i(x) = \|x - c_i\|^2 - r_i^2$$

when you want convex weighted cells, simpler bisectors, and more stable geometric infrastructure. Use the **spectral layer** only when the affinity graph carries information that plain Euclidean cells do not. And, regardless of mode, force the **hardware layer** to declare byte order, dtype, and alignment explicitly before any clustering begins. ³¹

The most important open questions are theoretical. The first is whether one can derive a **single joint objective** whose block updates include not only assignment and centroid means but also the radius closure

rule, without turning the method into a qualitatively different model. The second is whether there is a principled way to couple the radius update to spectral information—for example, by making r_i depend jointly on centroid spacing and local graph conductance. The third is whether multiplicative and power variants can be blended adaptively, choosing the geometry per cluster pair rather than globally. The fourth is reproducibility-oriented: what parts of a K3H4 pipeline should be required to be **bitwise stable** across hardware, and what parts only **numerically equivalent**? These questions remain open in this report because the underlying literature gives the necessary ingredients, but not this exact combined model. 32

The final synthesis is therefore simple. The conversation ties together if one sees it as a complete stack: **lambda calculus** gives the recursive form of the iteration, **K-means** gives the centroidal core, **Voronoi** gives the geometry, **inscribed hyperspheres** give scale, **endianness** gives the representation contract, and **harmonic or spectral analysis** gives the global mode structure when Euclidean partitioning is not enough. K3H4 is the name for that stack. 33

1 4 7 9 10 16 20 https://digicoll.lib.berkeley.edu/record/113015/files/math_s5_v1_article-17.pdf
https://digicoll.lib.berkeley.edu/record/113015/files/math_s5_v1_article-17.pdf?utm_source=chatgpt.com

2 11 31 32 Approximating Multiplicatively Weighted Voronoi Diagrams: Efficient Construction with Linear Size
<https://drops.dagstuhl.de/storage/00lipics/lipics-vol293-socg2024/LIPIcs.SoCG.2024.62/LIPIcs.SoCG.2024.62.pdf>
[utm_source=chatgpt.com](https://drops.dagstuhl.de/storage/00lipics/lipics-vol293-socg2024/LIPIcs.SoCG.2024.62/LIPIcs.SoCG.2024.62.pdf?utm_source=chatgpt.com)

3 5 14 15 29 2.3. Clustering — scikit-learn 1.9.0 documentation
https://scikit-learn.org/stable/modules/clustering.html?utm_source=chatgpt.com

6 17 Voronoi diagrams--a survey of a fundamental geometric data structure
https://www.wias-berlin.de/people/si/course/files/Aurenhammer91-Voronoi.pdf?utm_source=chatgpt.com

8 18 27 28 https://graphics.stanford.edu/courses/cs233-25-spring/ReferencedPapers/Luxburg07_spectral_clustering_tutorial_4488.pdf
https://graphics.stanford.edu/courses/cs233-25-spring/ReferencedPapers/Luxburg07_spectral_clustering_tutorial_4488.pdf?utm_source=chatgpt.com

12 Power Diagrams: Properties, Algorithms and Applications
https://www.cs.jhu.edu/~misha/Spring16/Aurenhammer87.pdf?utm_source=chatgpt.com

13 33 Programming in the λ -calculus 1 Nontermination 2 Recursion
https://www.cs.cornell.edu/courses/cs4110/2016fa/lectures/lecture16.pdf?utm_source=chatgpt.com

19 KMeans — scikit-learn 1.9.0 documentation
https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html?utm_source=chatgpt.com

21 23 struct — Interpret bytes as packed binary data — Python 3.14.6 documentation
https://docs.python.org/3/library/struct.html?utm_source=chatgpt.com

22 26 Byte-swapping — NumPy v2.4 Manual
https://numpy.org/doc/stable/user/byteswapping.html?utm_source=chatgpt.com

24 RFC 4506 - XDR: External Data Representation Standard
https://datatracker.ietf.org/doc/html/rfc4506?utm_source=chatgpt.com

25 **Data Alignment to Assist Vectorization**

https://www.intel.com/content/www/us/en/develop/articles/data-alignment-to-assist-vectorization.html?utm_source=chatgpt.com

30 **silhouette_score — scikit-learn 1.9.0 documentation**

https://scikit-learn.org/stable/modules/generated/sklearn.metrics.silhouette_score.html?utm_source=chatgpt.com